SkillGraph

AI-Adaptive Onboarding Engine

ARTPARK CodeForge Hackathon | IISc Bangalore | March 2026

COMPLETE PRODUCT REQUIREMENTS DOCUMENT

Implementation Reference for Claude Code — Read Entirely Before Writing One Line

Prize Pool	Build Window	Submission
₹30,000	**48 Hours**	**21 Mar 26, 11:59 PM IST**

SECTION 0 — HOW TO USE THIS DOCUMENT

READ THIS FIRST
This document is the single source of truth for building SkillGraph. Every decision has already been made.
Do NOT improvise. Do NOT add features not listed here. Do NOT use techniques marked as forbidden.
Build phases are in order. Follow them. Do not start frontend before backend logic passes terminal tests.
All data (skill lists, edges, courses) is embedded in this document as JSON. Use it verbatim.
If a judge asks any question about your system, the answer exists in Section 5 of this document.

1. Project Overview

1.1 What SkillGraph Does

SkillGraph takes a candidate resume and a target job description. It extracts skills, computes a mastery score per skill using a deterministic weighted formula, identifies the skill gap as a graph problem, and generates a personalized, dependency-aware learning path ordered by priority. Every course recommendation is pulled from a fixed catalog — the LLM never generates course names. Every path decision is explained by a deterministic reasoning trace.

1.2 One-Line Pitch

"We model skills as a dependency graph, compute gaps using a weighted mastery formula, and generate optimized learning paths via priority-aware topological traversal — grounded entirely in a fixed course catalog with zero hallucination."

1.3 What Makes This Win

Judging Criterion	Weight	How SkillGraph Addresses It
Technical Sophistication	20%	BKT-style mastery formula + graph traversal algorithm + deterministic priority scoring

Communication & Docs	20%	This PRD + clean README + structured 5-slide deck with slide-level algorithm explanations
Grounding & Reliability	15%	All recommendations from fixed catalog. LLM used only as classifier, never as generator
User Experience	15%	React Flow DAG visualization + reasoning panel + skill mastery view + mark-learned recompute
Cross-Domain Scalability	10%	O*NET-grounded skill taxonomy covering SWE and Data domains with identical algorithm
Reasoning Trace	10%	Deterministic per-node trace: required_by_JD, priority_score, unlocks_N_levels, mastery
Product Impact	10%	Demo shows 60%+ redundant module reduction for experienced hires; clear gap for juniors

2. Tech Stack (No Deviations)

Layer	Technology	Why This, Not Another
Backend Framework	FastAPI (Python)	Async, fast, auto-generates OpenAPI docs. No Django, no Flask.
Graph Engine	NetworkX (Python)	dag_longest_path_length, topological_generations, subgraph — all built in
LLM (Classifier)	Gemini 1.5 Flash via API	Free tier, fast, structured JSON output. Used ONLY for skill classification.
Frontend	Next.js 14 + TypeScript	App Router. No plain React. No Vite.
Graph Visualization	React Flow	Built for DAG rendering. Nodes, edges, custom node colors. No D3, no Cytoscape.
Skill Taxonomy	O*NET subset (hardcoded)	28 SWE skills + 27 Data skills, manually curated from O*NET occupational data
Course Catalog	Hardcoded JSON (~48 courses)	Fixed. Immutable. LLM cannot add to it. Ever.
HTTP Client (frontend)	Axios or native fetch	Simple. No Redux, no React Query needed.
Styling	Tailwind CSS	Utility-first. No custom CSS files.
Containerization	Docker + docker-compose	Optional but include it. Judges look for the Dockerfile.

FORBIDDEN TECHNOLOGIES — DO NOT USE

Embeddings / cosine similarity for JD matching — adds latency, breaks under pressure, provides no real gain
Vector databases (Chroma, Pinecone, FAISS) — unnecessary given fixed catalog
LangChain — overkill, adds abstraction that makes the algorithm invisible to judges
LangGraph — same reason as above; your graph is NetworkX, not LangGraph
Free-form LLM generation of course names or module titles — this is how hallucination enters
OpenAI API — cost, rate limits. Use Gemini Flash free tier.
Any frontend chart library (Chart.js, Recharts) — use React Flow for the skill graph only

3. Data Layer — The Foundation of Everything

Build this first. Test this before writing any business logic. If any edge is wrong, your path will look stupid. If any course covers the wrong skill, your recommendations will be incoherent.

3.1 SWE Domain — Skill List (28 skills)

Filename: `backend/data/skills_swe.json`

```
[
  "Python", "Data Structures", "Algorithms", "OOP", "SQL",
  "REST APIs", "Git", "Testing", "Linux/CLI", "Operating Systems",
  "Docker", "Computer Networks", "HTML/CSS", "JavaScript",
  "TypeScript", "React", "Node.js", "Databases (NoSQL)",
  "Cloud Basics", "CI/CD", "Security Basics", "GraphQL",
  "Microservices", "Caching (Redis)", "Message Queues",
  "Design Patterns", "System Design", "Clean Code"
]
```

3.2 Data Domain — Skill List (27 skills)

Filename: `backend/data/skills_data.json`

```
[
  "Python", "SQL", "Statistics", "Probability", "Linear Algebra",
  "NumPy", "Pandas", "Data Visualization", "Machine Learning Fundamentals",
  "Feature Engineering", "Supervised Learning", "Unsupervised Learning",
  "Model Evaluation", "Deep Learning", "Neural Networks", "NLP",
  "Computer Vision", "Data Engineering", "ETL Pipelines", "Spark",
  "Airflow", "MLOps", "Model Deployment", "Bayesian Methods",
  "Vector Databases", "LLMs", "Time Series"
]
```

3.3 SWE Dependency Edges (23 edges)

Filename: `backend/data/edges_swe.json` — Format: [source, target] where source is prerequisite for target

From (Prerequisite)	To (Depends on it)	Defense in 1 sentence
Python	Data Structures	Cannot implement DS without knowing the language
Python	OOP	OOP syntax is Python-specific
Data Structures	Algorithms	Algorithms operate on data structures
Algorithms	System Design	System design requires understanding algorithmic complexity
OOP	Design Patterns	Design patterns are OOP solutions
Design Patterns	Clean Code	Clean code principles build on structural pattern thinking
SQL	Databases (NoSQL)	NoSQL is understood by contrast with relational SQL
REST APIs	GraphQL	GraphQL is an alternative to REST; REST must be understood first
Git	CI/CD	CI/CD pipelines trigger on Git events
Testing	CI/CD	CI/CD runs tests automatically; must know testing first
Linux/CLI	Docker	Docker is CLI-first; requires comfort with terminal
Operating Systems	Linux/CLI	Linux CLI requires understanding of OS concepts
Docker	Cloud Basics	Cloud deployment uses containerized Docker images
Cloud Basics	Microservices	Microservices are deployed on cloud infrastructure
System Design	Microservices	Microservices is an advanced system design pattern
Microservices	Message Queues	Event-driven microservices use message queues
Microservices	Caching (Redis)	Redis is used within microservice architectures
Computer Networks	Security Basics	Security builds on network protocol knowledge
HTML/CSS	JavaScript	JS manipulates HTML/CSS — must know both first
JavaScript	TypeScript	TypeScript is a typed superset of JavaScript
JavaScript	React	React is a JavaScript framework
JavaScript	Node.js	Node.js runs JavaScript on server

Node.js	REST APIs	REST APIs are built with Node.js/ Express in JS stack

3.4 Data Domain Dependency Edges (28 edges)

Filename: `backend/data/edges_data.json`

From (Prerequisite)	To (Depends on it)	Defense in 1 sentence
Python	NumPy	NumPy is a Python library
Python	Pandas	Pandas is a Python library
Statistics	Probability	Probability theory is a branch of statistics
Statistics	Machine Learning Fundamentals	ML algorithms are grounded in statistical theory
Statistics	Time Series	Time series analysis requires statistical methods
Linear Algebra	Machine Learning Fundamentals	ML models use vectors, matrices, and transformations
Linear Algebra	Deep Learning	Neural networks are linear algebra operations at their core
Probability	Bayesian Methods	Bayesian methods are built on conditional probability
NumPy	Machine Learning Fundamentals	ML implementations use NumPy for matrix ops
Pandas	Feature Engineering	Feature engineering is done on DataFrames
Pandas	Data Visualization	Pandas integrates with matplotlib/ seaborn
Machine Learning Fundamentals	Supervised Learning	Supervised learning is a subtype of ML
Machine Learning Fundamentals	Unsupervised Learning	Unsupervised learning is a subtype of ML
Machine Learning Fundamentals	Deep Learning	DL requires understanding classical ML first
Feature Engineering	Model Evaluation	Feature quality is evaluated through model performance
Supervised Learning	Model Evaluation	Supervised models are evaluated with metrics
Model Evaluation	MLOps	MLOps tracks and automates model evaluation in production
Deep Learning	Neural Networks	Neural networks are the DL implementation unit
Neural Networks	NLP	NLP uses transformer neural networks

Neural Networks	Computer Vision	CV uses CNN neural network architectures
SQL	Data Engineering	Data engineering pipelines query and transform SQL data
Data Engineering	ETL Pipelines	ETL is the implementation of data engineering
ETL Pipelines	Spark	Spark is used for large-scale ETL
Spark	Airflow	Airflow orchestrates Spark jobs
MLOps	Model Deployment	Deployment is the last stage of MLOps
Model Deployment	Vector Databases	Deployed models often query vector databases
Vector Databases	LLMs	LLM applications use vector DBs for retrieval
Bayesian Methods	LLMs	LLM fine-tuning and RLHF use Bayesian reasoning

3.5 Course Catalog (48 courses)

Filename: `backend/data/courses.json` — difficulty: 1=Beginner, 2=Intermediate, 3=Advanced, 4=Expert

Course Name	Covers (Skills)	Difficulty	Domain
Python Fundamentals	Python	1	SWE
Data Structures & Algorithms	Data Structures, Algorithms	2	SWE
OOP with Python	OOP	2	SWE
SQL for Developers	SQL	1	SWE/Data
REST API Design & Development	REST APIs	2	SWE
Git & Version Control	Git	1	SWE
Software Testing Fundamentals	Testing	2	SWE
Linux & CLI Essentials	Linux/CLI, Operating Systems	1	SWE
Docker for Developers	Docker	2	SWE
System Design Fundamentals	System Design	3	SWE
Computer Networks Basics	Computer Networks, Security Basics	2	SWE
HTML & CSS Complete Guide	HTML/CSS	1	SWE
JavaScript Essentials	JavaScript	1	SWE

TypeScript for JS Developers	TypeScript	2	SWE
React.js Complete Guide	React	2	SWE
Node.js Backend Development	Node.js, REST APIs	2	SWE
NoSQL Databases	Databases (NoSQL)	2	SWE
Cloud Computing (AWS/GCP)	Cloud Basics	2	SWE
CI/CD Pipelines	CI/CD	2	SWE
GraphQL API Development	GraphQL	3	SWE
Microservices Architecture	Microservices, Design Patterns	3	SWE
Caching with Redis	Caching (Redis)	2	SWE
Message Queues & Event Systems	Message Queues	3	SWE
Clean Code & Design Patterns	Clean Code, Design Patterns	2	SWE
Statistics for Data Science	Statistics, Probability	2	Data
Linear Algebra for ML	Linear Algebra	2	Data
NumPy & Scientific Computing	NumPy	1	Data
Pandas for Data Analysis	Pandas, Data Visualization	1	Data
Feature Engineering Techniques	Feature Engineering	2	Data
ML Fundamentals (Scikit-Learn)	Machine Learning Fundamentals	2	Data
Supervised Learning	Supervised Learning, Model Evaluation	2	Data
Unsupervised Learning	Unsupervised Learning	3	Data
Deep Learning with PyTorch	Deep Learning, Neural Networks	3	Data
NLP with Transformers	NLP	3	Data
Computer Vision Fundamentals	Computer Vision	3	Data
Data Engineering with SQL	Data Engineering, ETL Pipelines	2	Data
Apache Spark for Big Data	Spark	3	Data
Airflow for Data Orchestration	Airflow	3	Data
MLOps & Model Deployment	MLOps, Model Deployment	3	Data
Bayesian Statistics	Bayesian Methods	3	Data
Time Series Analysis	Time Series	3	Data

Vector DBs & Embeddings	Vector Databases	3	Data
LLMs & Prompt Engineering	LLMs, NLP	3	Data
Data Visualization (Matplotlib/Seaborn)	Data Visualization	1	Data
Model Evaluation & Validation	Model Evaluation, Feature Engineering	2	Data

COURSE CATALOG RULES
A course's 'covers' array must ONLY contain skills that appear in skills_swe.json or skills_data.json
No course should cover skills from both domains unless it is tagged domain='SWE/Data'
Course names are final. Do not let the LLM generate or modify them.
If a skill has no matching course, flag it in the UI as 'No course available' — do not fabricate one
Difficulty is used for path ordering — within same priority, lower difficulty comes first

4. Core Algorithms — Every Formula Fully Defined

This is what judges will ask about. Know every formula cold. Every variable is computable from resume + JD text. Nothing requires data you won't have.

4.1 Step 1 — Skill Classification (LLM as Classifier ONLY)

What it does: Takes raw resume text and JD text. Uses LLM to classify which predefined skills are present. LLM does NOT generate skill names — it maps to your fixed list.

```
# backend/parser.py

SYSTEM_PROMPT = """
You are a skill extractor. Given resume text and a skill list, return ONLY a JSON array
of objects matching this schema: [{skill, mentions, in_recent_experience}]
Only include skills from the provided list. Do not invent new skills.
in_recent_experience = true if skill appears in last 2 jobs or last 3 years.
mentions = integer count of how many times the skill appears in the resume.
Return ONLY valid JSON. No explanation, no markdown.
"""

def classify_skills(resume_text: str, domain: str) -> list[dict]:
    skill_list = load_skills(domain)  # from skills_swe.json or skills_data.json
```

```
    prompt = f"Resume:
{resume_text}


Skill list: {skill_list}"
    response = call_gemini(SYSTEM_PROMPT, prompt)  # returns JSON
    return parse_json_safely(response)  # wrap in try/except

def classify_jd(jd_text: str, domain: str) -> dict:
    # Returns {required: [skills], preferred: [skills]}
    # Same approach — LLM classifies into predefined skill list
    ...
```

4.2 Step 2 — Mastery Score Computation

Formula:

```
# backend/mastery.py

WEIGHTS = { "frequency": 0.35, "recency": 0.35, "jd_match": 0.30 }
MASTERY_THRESHOLD = 0.6  # below this = skill gap

def compute_mastery(skill: str, resume_data: list[dict], jd_data: dict,
                    total_skills_mentioned: int) -> float:
    # Find skill in resume output
    skill_entry = next((s for s in resume_data if s["skill"] == skill), None)

    if skill_entry is None:
        return 0.0  # Not mentioned at all

    # frequency = normalized mention count (0.0 to 1.0)
    frequency = min(skill_entry["mentions"] / max(total_skills_mentioned, 1), 1.0)

    # recency = binary flag (no timestamp parsing needed)
    recency = 1.0 if skill_entry["in_recent_experience"] else 0.5

    # jd_match = importance weight from JD classification
    if skill in jd_data["required"]:
```

```
        jd_match = 1.0
    elif skill in jd_data["preferred"]:
        jd_match = 0.6
    else:
        jd_match = 0.0

    score = (WEIGHTS["frequency"] * frequency +
             WEIGHTS["recency"]   * recency +
             WEIGHTS["jd_match"]  * jd_match)

    return round(min(score, 1.0), 3)
```

MASTERY FORMULA — JUDGE DEFENSE
Q: What is frequency? A: Normalized mention count — mentions / total_skills_mentioned, capped at 1.0
Q: What is recency? A: Binary flag — 1.0 if the skill appears in the most recent job/section, 0.5 otherwise
Q: What is jd_match? A: 1.0 if required by JD, 0.6 if preferred, 0.0 if absent
Q: Why these weights? A: Balanced equal importance — frequency proves depth, recency proves currency, JD match proves relevance
Q: Why not embeddings for JD match? A: LLM classification into predefined list is more reliable and debuggable in a 48-hour build

4.3 Step 3 — Gap Identification

```
# backend/graph.py

def identify_gaps(all_skills: list[str], mastery_scores: dict,
                  jd_data: dict) -> set[str]:
    """
    A skill is a gap if:
      1. It is required or preferred in the JD
      2. AND its mastery score is below MASTERY_THRESHOLD (0.6)
    """
    jd_skills = set(jd_data["required"]) | set(jd_data["preferred"])
    gaps = {
        skill for skill in all_skills
        if skill in jd_skills and mastery_scores.get(skill, 0.0) < MASTERY_THRESHOLD
```

```
    }
    return gaps
```

4.4 Step 4 — Build Gap Subgraph

```
import networkx as nx

def build_skill_graph(domain: str) -> nx.DiGraph:
    edges = load_edges(domain)  # from edges_swe.json or edges_data.json
    G = nx.DiGraph()
    G.add_nodes_from(load_skills(domain))
    G.add_edges_from(edges)
    return G

def build_gap_subgraph(G: nx.DiGraph, gap_skills: set[str]) -> nx.DiGraph:
    """
    Include gap skills AND their prerequisite chains.
    A prerequisite of a gap skill is also included even if mastered,
    because you may need to verify the chain is complete.
    """
    nodes_to_include = set(gap_skills)
    for skill in gap_skills:
        ancestors = nx.ancestors(G, skill)
        nodes_to_include.update(ancestors)
    return G.subgraph(nodes_to_include).copy()
```

4.5 Step 5 — Priority Score

Formula: priority(s) = 0.4 * jd_importance(s) + 0.4 * downstream_depth(s) - 0.2 * mastery(s)

```
def compute_priority(skill: str, subgraph: nx.DiGraph,
                     jd_data: dict, mastery_scores: dict) -> float:
    # jd_importance: how critical is this to the JD
    if skill in jd_data["required"]:
        jd_importance = 1.0
    elif skill in jd_data["preferred"]:
        jd_importance = 0.6
    else:
        jd_importance = 0.0
```

```
    # downstream_depth: longest path from this node in the subgraph
    # Measures how many future skills this node unlocks
    try:
        downstream_depth = nx.dag_longest_path_length(
            subgraph.subgraph(
                {skill} | nx.descendants(subgraph, skill)
            )
        )
    except Exception:
        downstream_depth = 0

    mastery = mastery_scores.get(skill, 0.0)

    priority = (0.4 * jd_importance +
                0.4 * downstream_depth -
                0.2 * mastery)

    return round(priority, 4)
```

PRIORITY FORMULA — JUDGE DEFENSE
Q: Why downstream_depth over dependency_count? A: Count treats all successors equally. Depth captures nodes that unlock long chains of future skills — those are more valuable.
Q: Why is mastery negative? A: Higher mastery = less urgent. We subtract it to deprioritize skills the candidate already partially knows.
Q: Why 0.4 / 0.4 / 0.2? A: JD alignment and structural impact are equally important. Mastery adjustment is secondary.
Q: What is downstream_depth exactly? A: nx.dag_longest_path_length on the subgraph of {skill} union its descendants. This is a built-in NetworkX function.

4.6 Step 6 — Priority-Aware Topological Traversal

CRITICAL: This is NOT 'topological sort then re-sort by priority'. That breaks dependency constraints. This is a greedy frontier traversal.

```
def generate_learning_path(subgraph: nx.DiGraph,
                           priorities: dict[str, float],
                           mastery_scores: dict) -> list[str]:
    """
```

```
    Priority-aware topological traversal:
    At each step, pick the HIGHEST PRIORITY node from the current frontier.
    Frontier = nodes whose ALL prerequisites are already placed in the path.
    This ensures dependency constraints are NEVER violated.
    """
    in_degree = dict(subgraph.in_degree())
    path = []
    visited = set()

    # Initial frontier: nodes with no prerequisites in the subgraph
    frontier = [n for n, d in in_degree.items() if d == 0]

    while frontier:
        # Sort frontier by priority descending, then difficulty ascending as tiebreaker
        frontier.sort(key=lambda n: (-priorities[n], mastery_scores.get(n, 0.0)))

        # Pick highest priority node
        chosen = frontier.pop(0)
        path.append(chosen)
        visited.add(chosen)

        # Unlock successors whose all prerequisites are now visited
        for successor in subgraph.successors(chosen):
            if all(pred in visited for pred in subgraph.predecessors(successor)):
                frontier.append(successor)

    return path
```

TRAVERSAL — JUDGE DEFENSE

Q: Why is this path optimal? A: It respects all prerequisite constraints via frontier-based traversal and prioritizes high-impact skills using JD alignment + dependency depth + mastery gap.

Q: Could you get stuck in an infinite loop? A: No — each node is visited exactly once. The frontier only adds nodes when all their predecessors are already placed.

Q: What if two nodes have equal priority? A: Tiebreaker is mastery score ascending — the skill you know less about comes first.

Q: Why not just use nx.topological_sort? A: topological_sort gives one valid ordering but not the optimal one. Our greedy selection within valid options gives a goal-directed path.

4.7 Step 7 — Course Recommendation

```
def recommend_course(skill: str, gap_skills: set[str], catalog: list[dict]) -> dict:
    """
    For each skill in the path, find the course that:
    1. Covers that skill (primary filter)
    2. Maximizes coverage of OTHER gap skills (bonus)
    3. Lower difficulty preferred if priority scores are tied
    """
    candidates = [c for c in catalog if skill in c["covers"]]

    if not candidates:
        return {"name": "No course available", "covers": [skill], "difficulty": 0}

    def score_course(course):
        gap_coverage = sum(1 for s in course["covers"] if s in gap_skills)
        return (gap_coverage, -course["difficulty"])  # more gap coverage, easier =
better

    return max(candidates, key=score_course)
```

4.8 Step 8 — Deterministic Reasoning Trace

```
# backend/reasoning.py

def generate_trace(skill: str, path_position: int, subgraph: nx.DiGraph,
                   priorities: dict, mastery_scores: dict,
                   jd_data: dict) -> dict:

    required = skill in jd_data["required"]
    preferred = skill in jd_data.get("preferred", [])
    mastery = mastery_scores.get(skill, 0.0)
    priority = priorities[skill]
    depth = nx.dag_longest_path_length(
        subgraph.subgraph({skill} | nx.descendants(subgraph, skill))
```

```
    ) if nx.descendants(subgraph, skill) else 0
    unlocks = list(subgraph.successors(skill))

    # Build reason string from deterministic logic — no LLM
    reasons = []
    if required:
        reasons.append("explicitly required by the Job Description")
    elif preferred:
        reasons.append("listed as preferred in the Job Description")
    if depth >= 2:
        reasons.append(f"unlocks {depth} downstream skill levels ({', '.join(unlocks[:3])}...)")
    elif unlocks:
        reasons.append(f"directly unlocks: {', '.join(unlocks)}")
    if mastery < 0.3:
        reasons.append(f"current mastery is low ({mastery:.2f})")
    if path_position == 0:
        reasons.append("no prerequisites — valid starting point")

    if reasons:
        reason_str = "Selected because: " + "; ".join(reasons) + "."
    else:
        reason_str = "Required dependency for downstream skills."

    return {
        "skill": skill,
        "position": path_position + 1,
        "mastery": mastery,
        "priority_score": priority,
        "required_by_jd": required,
        "preferred_by_jd": preferred,
        "unlocks": unlocks,
        "downstream_depth": depth,
        "reason": reason_str
    }
```

4.9 Step 9 — Adaptive Recomputation (Mark Learned)

When user marks a skill as learned, update their mastery to 1.0 for that skill and rerun Steps 3-8. This is what makes the system 'adaptive'.

```
@app.post("/recompute")
def recompute_path(request: RecomputeRequest):
    # request contains: original_state + skill_marked_as_learned
    updated_mastery = {**request.mastery_scores,
                       request.learned_skill: 1.0}
    # Re-run gap identification → subgraph → priorities → traversal
    return run_pipeline(request.domain, request.resume_skills,
                        request.jd_data, updated_mastery)
```

5. API Design — Endpoints & Schemas

Method	Endpoint	Input	Output
POST	/parse	{ resume_text, jd_text, domain }	{ skills_with_mastery, jd_required, jd_preferred }
POST	/pathway	{ domain, resume_skills, jd_data, mastery_scores }	{ path[], reasoning[], course_map{} }
POST	/recompute	{ ...state, learned_skill: str }	{ path[], reasoning[], course_map{} }
GET	/graph/{domain}	domain = 'swe' or 'data'	{ nodes[], edges[] } for React Flow
GET	/catalog/{domain}	domain = 'swe' or 'data'	Full course catalog array
GET	/health	None	{ status: 'ok' }

5.1 Full /pathway Response Schema

```
{
  "path": ["Python", "NumPy", "ML Fundamentals", ...],

  "reasoning": [
    {
      "skill": "Python",
      "position": 1,
      "mastery": 0.21,
      "priority_score": 0.88,
      "required_by_jd": true,
      "preferred_by_jd": false,
      "unlocks": ["NumPy", "Pandas"],
```

```
      "downstream_depth": 5,
      "reason": "Selected because: explicitly required by the JD; unlocks 5 downstream skill levels."
    },
    ...
  ],

  "course_map": {
    "Python": { "name": "Python Fundamentals", "covers": ["Python"], "difficulty": 1 },
    "NumPy":  { "name": "NumPy & Scientific Computing", "covers": ["NumPy"], "difficulty": 1 },
    ...
  },

  "mastery_scores": { "Python": 0.21, "NumPy": 0.0, ... },
  "gap_count": 8,
  "domain": "data"
}
```

6. Frontend — Components & Layout

6.1 Page Layout

Single page. Three-panel layout after upload + submit.

LEFT PANEL	CENTER PANEL	RIGHT PANEL
SkillPanel.tsx	**GraphView.tsx**	**PathPanel.tsx**
• All skills in domain	• React Flow DAG	• Ordered module list
• Mastery bar per skill	• Green = mastered	• Course per skill
• Color: green/yellow/red	• Yellow = partial	• Reasoning accordion
• Gap skills highlighted	• Red = critical gap	• Mark Learned button
	• Orange = selected path	

6.2 Component File List

File	Purpose	Key Props / State
app/page.tsx	Root — orchestrates state and API calls	stage: upload|loading|results

components/Upload.tsx	Resume + JD text areas + domain selector + Submit	onSubmit(resumeText, jdText, domain)
components/SkillPanel.tsx	Left panel: all skills with mastery bars	skills, mastery_scores, gap_skills
components/GraphView.tsx	Center: React Flow DAG with colored nodes	nodes, edges, path (for highlighting)
components/PathPanel.tsx	Right: ordered path with courses + reasoning	path, reasoning, course_map, onMarkLearned
components/ReasoningCard.tsx	Expandable card per skill with trace details	trace object from /pathway response
lib/api.ts	All fetch calls to FastAPI	parse(), getPathway(), recompute()

6.3 React Flow Node Colors

```
// In GraphView.tsx
const getNodeColor = (skill: string, mastery: number, path: string[]) => {
  if (path.includes(skill)) return "#C84B1E"  // orange = selected learning path
  if (mastery >= 0.8)        return "#1B7A48"  // green = mastered
  if (mastery >= 0.4)        return "#F5A623"  // yellow = partial
  if (mastery > 0.0)         return "#E05A4E"  // red = critical gap
  return "#9B9B9B"                             // gray = not in resume at all
}
```

7. Repository File Structure

```
skillgraph/
├── backend/
│   ├── main.py              # FastAPI app, all routes
│   ├── parser.py            # Gemini skill classifier
│   ├── mastery.py           # Mastery formula (Section 4.2)
│   ├── graph.py             # Graph build + traversal (Section 4.4-4.6)
│   ├── reasoning.py         # Deterministic trace (Section 4.8)
│   ├── courses.py           # Course recommendation (Section 4.7)
│   ├── models.py            # Pydantic request/response schemas
│   ├── data/
│   │   ├── skills_swe.json  # 28 SWE skills (Section 3.1)
│   │   ├── skills_data.json # 27 Data skills (Section 3.2)
│   │   ├── edges_swe.json   # 23 SWE edges (Section 3.3)
│   │   ├── edges_data.json  # 28 Data edges (Section 3.4)
│   │   └── courses.json     # 48 courses (Section 3.5)
```

```
│   └── requirements.txt
├── frontend/
│   ├── src/
│   │   ├── app/
│   │   │   ├── page.tsx
│   │   │   └── layout.tsx
│   │   ├── components/
│   │   │   ├── Upload.tsx
│   │   │   ├── SkillPanel.tsx
│   │   │   ├── GraphView.tsx
│   │   │   ├── PathPanel.tsx
│   │   │   └── ReasoningCard.tsx
│   │   └── lib/
│   │       └── api.ts
│   ├── package.json
│   └── next.config.js
├── Dockerfile
├── docker-compose.yml
└── README.md
```

8. Build Order — Do Not Deviate

THE RULE

Do NOT start Phase 2 until Phase 1 data passes manual inspection.

Do NOT start Phase 4 (frontend) until Phase 2 passes terminal tests.

Do NOT touch demo polish until everything else works end-to-end.

Phase	Task	Est. Time	Done When
Phase 1	Create all JSON data files (skills, edges, courses)	1-2 hrs	Manual inspection: every edge is defensible, every course covers real skills
Phase 2	Build backend: parser, mastery, graph, traversal, reasoning, courses, main.py routes	7-9 hrs	Terminal test: paste demo input, get correct JSON output
Phase 3	Build reasoning trace generator. Test determinism: same input = same output every time.	2 hrs	Each skill in demo path has a clear, accurate reason string

Phase 4	Build frontend: Upload -> call API -> render 3 panels + React Flow DAG	4-5 hrs	Demo input 1 renders correctly in browser
Phase 5	Add Mark Learned -> / recompute -> update graph + path	1-2 hrs	Clicking Mark Learned updates the path visibly
Phase 6	Demo preparation: run both demo personas, verify outputs, prepare fallback	1 hr	Both demos produce expected paths. No surprises.
Phase 7	README + Docker + cleanup	1 hr	README has setup instructions, architecture overview, algorithm explanation

9. Fixed Demo Inputs — Know These Cold

Use ONLY these inputs during the demo. Do not improvise with unknown resumes. Know the expected outputs before you present.

Demo 1: Junior SWE → Senior SWE

Field	Value
Domain	SWE
Resume skills	Python (5 mentions, recent), HTML/CSS (3 mentions, recent), JavaScript (4 mentions, recent), Git (2 mentions, recent), SQL (1 mention, not recent), OOP (2 mentions, not recent)
JD — Required	System Design, Docker, CI/CD, Microservices, Testing, TypeScript
JD — Preferred	GraphQL, Caching (Redis), Cloud Basics
Expected gap skills	System Design, Docker, CI/CD, Microservices, Testing, TypeScript, GraphQL, Caching (Redis), Cloud Basics
Expected path start	Testing → Data Structures → Algorithms → TypeScript → Linux/CLI → Docker → CI/CD → Cloud Basics → System Design → Microservices → Caching (Redis) → GraphQL
Story to tell	This hire knows frontend JS but was never in a production backend system. The path skips Python basics, skips HTML/CSS, and goes straight to the production-readiness gap.

Demo 2: Junior Data Analyst → ML Engineer

Field	Value
Domain	Data
Resume skills	SQL (6 mentions, recent), Pandas (3 mentions, recent), Python (2 mentions, recent), Data Visualization (2 mentions, not recent)

JD — Required	Machine Learning Fundamentals, Deep Learning, Feature Engineering, MLOps, Model Deployment, NumPy
JD — Preferred	NLP, Vector Databases, LLMs
Expected gap skills	Machine Learning Fundamentals, Deep Learning, Feature Engineering, MLOps, Model Deployment, NumPy, NLP, Vector Databases, LLMs
Expected path start	Statistics → Linear Algebra → NumPy → ML Fundamentals → Supervised Learning → Feature Engineering → Model Evaluation → Deep Learning → Neural Networks → MLOps → Model Deployment → Vector Databases → NLP → LLMs
Story to tell	This hire can query data but has never built a model. The path preserves their SQL/Pandas mastery and builds upward. The graph makes the dependency chain visual — statistics must come before ML, linear algebra before deep learning.

10. Judge Q&A — Memorize These Answers

These are the questions you will be asked. If you hesitate, you lose credibility. Say these answers exactly.

Question	Exact Answer
Why is this path optimal?	It respects all prerequisite constraints via frontier-based topological traversal, and at each step selects the highest-priority skill based on JD alignment, downstream dependency depth, and current mastery gap. No valid dependency is ever violated.
How do you ensure no hallucination?	All course names and modules are selected from a fixed JSON catalog. The LLM is used only to classify which predefined skills appear in the resume and JD — it cannot generate new skill names or course titles. If a skill has no matching course, the system flags it explicitly.
What makes this adaptive?	Adaptivity comes from dynamically computing mastery scores per user input and recomputing the skill graph traversal accordingly. When a user marks a skill as learned, mastery updates to 1.0 and the full pathway reruns — what was a gap is now mastered, and the path adjusts.
How do you compute mastery?	Three weighted terms: normalized mention frequency from resume (0.35), binary recency flag — did it appear in recent experience (0.35), and JD match weight — required = 1.0, preferred = 0.6 (0.30). The formula is deterministic and fully computable from resume text.

How does the reasoning trace work?	It is entirely deterministic — no LLM involved. For each skill in the path, we compute: required by JD (boolean), downstream depth (nx.dag_longest_path_length), current mastery score, and which skills it directly unlocks. These facts are assembled into a human-readable reason string.
How does this scale to other job domains?	The system is parameterized by domain. Each domain has its own skill list and edge graph loaded from JSON. Adding a new domain requires defining a skill list, prerequisite edges, and course mappings — the algorithm is identical. We demonstrate SWE and Data; Operations or Finance would follow the same structure.
Why not use embeddings for skill matching?	We use LLM-based classification into a predefined skill list instead. This is more reliable — embeddings can produce unexpected similarity scores for technical skill names that don't behave semantically. Classification into a fixed list is debuggable, testable, and produces no ambiguous matches.
How do you handle prerequisites the user already knows?	If mastery >= 0.6 for a prerequisite skill, it is not included in the gap subgraph and therefore does not appear in the learning path. The path only includes what the user genuinely needs. Mastered skills appear in the graph visualization as green nodes for reference.

11. Hard Rules — Zero Exceptions

DO NOT DO ANY OF THESE
Let the LLM generate course names, skill names, or any catalog content
Use embeddings or cosine similarity anywhere in the pipeline
Use LangChain or LangGraph — NetworkX IS the graph engine
Call topological_sort and then re-sort by priority (breaks constraints)
Claim the system works for 900+ O*NET roles — say prototype covers SWE + Data
Add features not in this document during the hackathon
Start frontend before backend terminal tests pass
Demo with a resume you have not already tested
Mention BKT to judges without following up with the exact formula
Leave any TODO comments in the submitted code

12. README.md Outline

Communication is 20% of the rubric. The README must be excellent. Use this structure exactly.

```
# SkillGraph — AI-Adaptive Onboarding Engine
```

What It Does (2 sentences)

Live Demo

[Link if deployed] | [Video walkthrough link]

Architecture

[Diagram or ASCII art of: Resume/JD → Parser → Mastery Scorer → Gap Identifier → Skill Graph (NetworkX) → Priority Traversal → Course Mapper → Frontend]

Skill-Gap Analysis Logic

Explain: mastery formula, gap filter, priority score, traversal algorithm.

Include the actual formulas. This is what judges read.

Tech Stack

Backend: FastAPI, NetworkX, Google Gemini API

Frontend: Next.js, React Flow, Tailwind CSS

Data: O*NET-grounded skill taxonomy (SWE + Data domains)

Setup Instructions

```
# Backend
cd backend && pip install -r requirements.txt
export GEMINI_API_KEY=your_key
uvicorn main:app --reload

# Frontend
cd frontend && npm install && npm run dev
```

Datasets & Models Used

- O*NET Database: skill taxonomy and prerequisite structure
- Google Gemini 1.5 Flash: resume/JD skill classification (classifier only)
- Course catalog: manually curated from NPTEL and Coursera public data

Adaptive Logic (Original Implementation)

[Explain that the graph construction, mastery formula, priority scoring,

```
and traversal are original implementations — not a library call]
```

13. Five-Slide Deck Content

Slide	Title	Must Include
1	Solution Overview	One-line pitch. The pain (static onboarding = wasted time or overwhelmed hires). How SkillGraph solves it: personalized paths from skill graph + mastery formula. Show the 3-panel UI screenshot.
2	Architecture & Workflow	Data flow diagram: Resume+JD → LLM Classifier → Mastery Scorer → Gap Subgraph → Priority Traversal → Course Mapper → React Flow UI. Each arrow labeled with what's passing through it.
3	Tech Stack & Models	Backend: FastAPI, NetworkX, Gemini 1.5 Flash (classifier only). Frontend: Next.js, React Flow. Data: O*NET skill taxonomy. Emphasize: Gemini = classifier, not generator. NetworkX = graph engine. No embeddings.
4	Algorithms & Training	Full mastery formula with variable definitions. Priority formula. Traversal algorithm (frontier-based greedy, NOT naive topo sort). Show the reasoning trace JSON for one skill. This is the technical credibility slide.
5	Datasets & Metrics	O*NET (skill taxonomy), manually curated course catalog (~48 courses, 2 domains). Metrics: % redundant modules eliminated per demo (measure it), path length vs. naive all-skills path, reasoning trace completeness (100% deterministic). Note: prototype covers SWE + Data domains.

FINAL REMINDER BEFORE YOU START CODING

1. Create the JSON data files first. Do not write a single line of Python until skills_swe.json, edges_swe.json, and courses.json exist and are correct.

2. Test the backend in the terminal before building any frontend.

3. Prepare your two demo inputs before the UI is even done.

4. The system must explain itself. Every path node has a reason. Every formula has a defense.

5. Stop iterating on strategy. This document has every decision. Build it.